



Data Handling with Python

CSV Files & the csv Module

Table of Contents

1. Introduction to CSV Files
 - 1.1 What is a CSV File?
 - 1.2 Why CSV Files are Used
 - 1.3 Structure of a CSV File
 - 1.4 Real-World Examples of CSV Files
2. Reading CSV Files
 - 2.1 Reading CSV Files Using Python's open() Function
 - 2.2 Reading CSV Files Using the csv Module
 - 2.3 Reading a CSV File Row by Row
 - 2.4 Accessing Specific Columns
3. Writing CSV Files
 - 3.1 Creating a New CSV File
 - 3.2 Appending Data to an Existing CSV File
 - 3.3 Writing Multiple Rows at Once
4. Using the csv Module
 - 4.1 Why Use the csv Module?
 - 4.2 csv.reader — Reading Data
 - 4.3 csv.writer — Writing Data
5. Working with DictReader and DictWriter
 - 5.1 csv.DictReader
 - 5.2 csv.DictWriter
 - 5.3 When to Use Dict vs Regular Reader/Writer
6. Handling Delimiters and Headers
 - 6.1 What is a Delimiter?
 - 6.2 Custom Delimiters in csv.reader
 - 6.3 Handling Headers
 - 6.4 Skipping the Header Row
7. Dealing with CSV Datasets
 - 7.1 What is a Dataset?

7.2 Loading a Real Dataset

7.3 Filtering Rows Based on Conditions

7.4 Sorting and Aggregating Data

7.5 Writing a Cleaned Dataset to a New File

Chapter Summary

ETHANS
CLOUD & AI ACADEMY



Chapter 1

Introduction to CSV Files

Welcome to the chapter on Data Handling with Python! In this chapter, you will learn one of the most important skills in programming — reading and writing data from files. We will focus on a specific and very popular file format called **CSV**, which stands for Comma-Separated Values.

By the end of this chapter, you will understand what CSV files are, how they are structured, and why they are used so widely in the real world — from spreadsheets to databases to business reports.

1.1 What is a CSV File?

CSV File

A plain text file that stores data in a table-like format. Each row in the file represents one record (like one row in a spreadsheet), and each piece of data within that row is separated by a comma.

Think of a CSV file as a simplified version of a spreadsheet. It has rows and columns, but without any formatting, formulas, or colours — just raw data stored as plain text.



Real-World Analogy — The Student Register

Imagine a school keeps a paper register of student information with the following columns:

Name, Age, Course, Grade

Each row in that register is one student's record. A CSV file works exactly the same way — each row is one record, and commas separate each piece of information within that record.

A CSV file has the **.csv** file extension. You can open it in Microsoft Excel, Google Sheets, a text editor like Notepad, and of course — with Python.

1.2 Why CSV Files are Used

CSV files are one of the most popular ways to store and share data. Here is why they are so widely used:

Reason	Explanation
Simple and lightweight	CSV files are plain text — they take up very little space and do not require any special software to open.

Reason	Explanation
Universal compatibility	Every spreadsheet program, database, and programming language can read CSV files. Data can be shared between different systems easily.
Human-readable	You can open a CSV file in Notepad and read the data directly — unlike binary formats like .xlsx or .db.
Easy to create and edit	You can create a CSV file using Python, Excel, Google Sheets, or even Notepad — no special tools are needed.
Ideal for data exchange	Organizations use CSV to export data from one system (like a database) and import it into another (like an analytics tool).

1.3 Structure of a CSV File

A CSV file follows a simple and consistent structure. Let us look at an example step by step.

Suppose a company wants to store employee records. Here is what the CSV file would look like when opened in a text editor:

```
Name,Department,Salary,City
Alice Sharma,Engineering,75000,Ahmedabad
Bob Mehta,Marketing,62000,Mumbai
Carol Patel,HR,58000,Pune
David Singh,Engineering,82000,Bangalore
Eva Nair,Finance,70000,Chennai
```

Figure 1.1 — A sample employee CSV file (as seen in a text editor)

Let us break down what each part of the file means:

Element	What It Is	Example from the File
Header Row	The first row of a CSV file. It contains column names that describe what data each column holds.	Name,Department,Salary,City
Data Row	Each row after the header row. It contains one complete record.	Alice Sharma,Engineering,75000,Ahmedabad
Field	Each individual value in a row, separated by commas.	Alice Sharma Engineering 75000 Ahmedabad
Delimiter	The character that separates fields from one another. In a standard CSV, this is a comma (,).	The , between each value

**Note**

Not all CSV files use a comma as the separator. Some use semicolons (;), tabs (\t), or pipes (|) as the delimiter. These are still referred to as 'CSV files' in common usage, even though technically they use a different separator. You will learn how to handle these in Chapter 6.

1.4 Real-World Examples of CSV Files

CSV files are used across almost every industry. Here are some practical examples you are likely to encounter:

Industry / Use Case	What is Stored in the CSV
School / University	Student name, roll number, marks, attendance, course enrolled
E-Commerce	Product ID, product name, price, stock quantity, category
Hospital / Healthcare	Patient ID, name, age, diagnosis, treatment date, doctor assigned
Banking / Finance	Account number, transaction date, amount, transaction type, balance
HR / Payroll	Employee ID, name, department, designation, salary, joining date
Weather Data	Date, location, temperature, humidity, wind speed, rainfall
Sales Reports	Order ID, customer name, product, quantity, unit price, total amount

**CSV in the Real World**

When you download your bank statement as a file, it is usually available in CSV format. When a data analyst receives sales data from a business, it almost always comes as a CSV file. When a data science project needs historical stock prices or weather data, that data is typically stored and shared in CSV format.

Learning to work with CSV files in Python is an essential skill for anyone entering the fields of data science, business analytics, software development, or automation.

Chapter 2

Reading CSV Files

Now that you understand what CSV files are, it is time to learn how to read them using Python. Reading a file means loading its contents into your program so that you can work with the data — display it, process it, filter it, or analyze it.

In this chapter, you will learn two ways to read CSV files: using Python's built-in **open()** function (the manual approach), and using Python's **csv module** (the professional approach).

2.1 Reading CSV Files Using Python's open() Function

The most basic way to read any file in Python is by using the built-in **open()** function. This function opens a file and gives you access to its contents. Let us start with this approach so you understand the fundamentals before moving to the csv module.

First, let us assume we have a file called **students.csv** saved on our computer with the following content:

```
Name,Age,Course,Grade
Alice Sharma,21,Computer Science,A
Bob Mehta,22,Data Science,B
Carol Patel,20,Computer Science,A
David Singh,23,AI & ML,A
Eva Nair,21,Data Science,B
```

Figure 2.1 — Content of students.csv

Now, let us read this file using **open()**:

```
# Reading a CSV file using open()

file = open('students.csv', 'r') # Open in read mode ('r')
content = file.read()           # Read all content as a single string
print(content)
file.close()                    # Always close the file after reading
```

Figure 2.3 — Reading a CSV file using the open() function

Output

```
Name,Age,Course,Grade
```

```
Alice Sharma,21,Computer Science,A  
Bob Mehta,22,Data Science,B  
Carol Patel,20,Computer Science,A  
David Singh,23,AI & ML,A  
Eva Nair,21,Data Science,B
```

While this works, the output is just raw text — a single long string. It does not separate the rows and columns for you automatically. To actually work with individual rows, you would need to manually split by newlines and then split each row by commas. This becomes complex very quickly.



Pro Tip

Always use the **with** statement when opening files. It automatically closes the file when you are done — even if an error occurs. This is considered best practice in Python.

```
with open('students.csv', 'r') as file:  
    content = file.read()  
    print(content)  
# File is automatically closed here — no need to call file.close()
```

2.2 Reading CSV Files Using the csv Module

csv Module

A built-in Python library that provides tools specifically designed to read and write CSV files. It handles commas inside text, special characters, and different delimiters automatically — without you needing to write that logic yourself.

Python includes a built-in module called **csv** specifically for working with CSV files. To use it, you must first import it at the top of your program. Since it is a built-in module, you do not need to install anything.

```
import csv # This line makes the csv module available in your program
```

Figure 2.4 — Importing the csv module

2.3 Reading a CSV File Row by Row

The most common way to read a CSV file using the **csv** module is with **csv.reader**. This reads the file one row at a time, and each row is returned as a **list** of values.

```
import csv  
  
# Reading students.csv row by row
```

```
with open('students.csv', 'r') as file:

    reader = csv.reader(file)    # Create a reader object

    for row in reader:          # Each 'row' is a list of values
        print(row)
```

Figure 2.6 — Reading a CSV file row by row using `csv.reader`

Output

```
['Name', 'Age', 'Course', 'Grade']
['Alice Sharma', '21', 'Computer Science', 'A']
['Bob Mehta', '22', 'Data Science', 'B']
['Carol Patel', '20', 'Computer Science', 'A']
['David Singh', '23', 'AI & ML', 'A']
['Eva Nair', '21', 'Data Science', 'B']
```

Notice that each row is now a Python **list**. The header row (Name, Age, Course, Grade) is also read as a list. Each value in the list is a string.



How `csv.reader` Works

When you call `csv.reader(file)`, Python creates a reader object that:

1. Reads the file one line at a time
2. Splits each line at every comma to produce a list
3. Handles edge cases automatically — such as commas inside quoted text like "Smith, John"

This is why `csv.reader` is much better than splitting manually with `.split(',')`

2.4 Accessing Specific Columns

Since each row is a list, you can access individual columns using their index position (remember — Python lists start at index 0).

```
import csv

# Print only the student names and their grades
with open('students.csv', 'r') as file:
    reader = csv.reader(file)

    next(reader)  # Skip the header row (we don't want to print 'Name, Grade')
```



```
for row in reader:
    name = row[0] # Index 0 = Name column
    grade = row[3] # Index 3 = Grade column
    print(f'Student: {name} | Grade: {grade}')
```

Figure 2.7 — Accessing specific columns by index

Output

```
Student: Alice Sharma | Grade: A
Student: Bob Mehta   | Grade: B
Student: Carol Patel | Grade: A
Student: David Singh | Grade: A
Student: Eva Nair    | Grade: B
```

The **next(reader)** call on line 8 moves the reader past the first row (the header) so that we only process the actual data rows. This is a very common pattern when working with CSV files.



Note

All values read from a CSV file using `csv.reader` are **strings**, even if they look like numbers. For example, the age '21' is stored as the string '21', not the integer 21.

To use them in calculations, you must convert them: **`int(row[1])`** or **`float(row[1])`**

Chapter 3

Writing CSV Files

So far, you have learned how to read data from an existing CSV file. Now you will learn how to write data to a CSV file — creating new files, adding new records, and updating existing files.

Writing CSV files is essential when you want to save the results of your program — such as a list of processed records, a filtered dataset, or a generated report — so it can be used later or shared with others.

3.1 Creating a New CSV File

To create a new CSV file and write data into it, you use **csv.writer** along with the **open()** function, but this time in **write mode ('w')** instead of read mode.

```
import csv

# Data to write — each inner list is one row
employee_data = [
    ['Name', 'Department', 'Salary', 'City'],      # Header row
    ['Alice Sharma', 'Engineering', 75000, 'Ahmedabad'],
    ['Bob Mehta', 'Marketing', 62000, 'Mumbai'],
    ['Carol Patel', 'HR', 58000, 'Pune'],
    ['David Singh', 'Engineering', 82000, 'Bangalore'],
]

# Writing to a new CSV file
with open('employees.csv', 'w', newline='') as file:
    writer = csv.writer(file)      # Create a writer object
    writer.writerows(employee_data) # Write all rows at once

print('File created successfully!')
```

Figure 3.1 — Creating a new CSV file using csv.writer

Output

File created successfully!

! Important

Always include **newline=""** when opening a file for writing in Python on Windows. Without it, Python adds an extra blank line after every row in the CSV file.

Correct: with open('file.csv', 'w', newline='') as file:

Incorrect: with open('file.csv', 'w') as file: ← creates blank lines between rows

3.2 Appending Data to an Existing CSV File

When you open a file in write mode ('w'), it **overwrites** the entire file. If you want to add new rows to an existing CSV file without deleting the current data, you must open it in **append mode ('a')**.

```
import csv

# New employees to add
new_employees = [
    ['Eva Nair', 'Finance', 70000, 'Chennai'],
    ['Farhan Khan', 'Engineering', 78000, 'Hyderabad'],
]

# Appending to the existing employees.csv
with open('employees.csv', 'a', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(new_employees) # Add only the new rows

print('New employees added successfully!')
```

Figure 3.3 — Appending rows to an existing CSV file

File Mode	What It Does
'w' — Write Mode	Creates a new file. If the file already exists, it is completely overwritten and all existing data is lost.
'a' — Append Mode	Opens the existing file and adds new data at the end. Existing data is kept intact.
'r' — Read Mode	Opens the file for reading only. You cannot write to the file in this mode.

3.3 Writing Multiple Rows at Once

The `csv.writer` object provides two methods for writing data:

Method	What It Does	Example
<code>writer.writerow(row)</code>	Writes a single row. The argument is one list.	<code>writer.writerow(['Alice', 21, 'CSE'])</code>
<code>writer.writerows(rows)</code>	Writes multiple rows at once. The argument is a list of lists.	<code>writer.writerows(all_data)</code>

```
import csv

with open('products.csv', 'w', newline='') as file:
    writer = csv.writer(file)

    # Write the header row individually
    writer.writerow(['Product', 'Category', 'Price', 'Stock'])

    # Write multiple data rows at once
    writer.writerows([
        ['Laptop', 'Electronics', 55000, 120],
        ['Notebook', 'Stationery', 40, 5000],
        ['Headphones', 'Electronics', 1200, 300],
        ['Pen Drive', 'Electronics', 450, 750],
    ])

print('products.csv has been created.')
```

Figure 3.4 — Using `writerow()` and `writerows()` together

Chapter 4

Using the csv Module

You have already seen the csv module in action in the previous chapters. In this chapter, we will take a deeper look at what the csv module offers, why it is the preferred tool for working with CSV files, and explore its key components in detail.

4.1 Why Use the csv Module?

You might wonder — if you can read a CSV file using `open()` and manually split by commas, why bother with a separate module?

The answer is: CSV files are more complex than they appear. Consider these situations:

Problem with Manual Splitting	How csv Module Handles It
A name like "Smith, John" contains a comma inside it. If you split by comma, it breaks incorrectly.	csv module understands quoted fields and correctly handles commas inside quotes.
Some CSV files use semicolons or tabs instead of commas.	csv module supports custom delimiters via a single parameter.
Lines may end differently on Windows vs Mac/Linux.	csv module handles different line endings automatically.
Some fields may have leading or trailing spaces.	csv module can strip whitespace automatically.
Numbers with decimals (3,14 in European format) may be confused with separators.	csv module handles different locale formats when configured.

In short — the csv module removes complexity, handles edge cases, and makes your code cleaner and more reliable.

4.2 csv.reader — Reading Data

`csv.reader` is the core tool for reading CSV files. When you pass a file object to `csv.reader`, it returns a reader object that you can loop over to get each row as a list.

```
import csv

# Full example — reading and processing a CSV
```

```

with open('students.csv', 'r') as file:
    reader = csv.reader(file)

    header = next(reader) # Read the header row separately
    print('Columns:', header)
    print('-' * 45)

    for row in reader:
        name = row[0]
        age = int(row[1]) # Convert to integer for calculations
        course = row[2]
        grade = row[3]
        print(f'{name:20} | Age: {age} | {course:20} | Grade: {grade}')

```

Figure 4.1 — Using `csv.reader` to read and process data**Output**

Columns: ['Name', 'Age', 'Course', 'Grade']

```

-----
Alice Sharma   | Age: 21 | Computer Science | Grade: A
Bob Mehta     | Age: 22 | Data Science    | Grade: B
Carol Patel   | Age: 20 | Computer Science | Grade: A
David Singh   | Age: 23 | AI & ML         | Grade: A
Eva Nair      | Age: 21 | Data Science    | Grade: B

```

4.3 csv.writer — Writing Data

csv.writer is the tool for writing data to CSV files. It creates a writer object that accepts your data and formats it properly as CSV — adding commas, handling quotes, and managing line endings automatically.

```

import csv

# Sales data to write
sales_records = [
    ['Order ID', 'Customer', 'Product', 'Quantity', 'Amount'],
    ['ORD-001', 'Alice Sharma', 'Laptop', 1, 55000],
    ['ORD-002', 'Bob Mehta', 'Headphones', 2, 2400],
    ['ORD-003', 'Carol Patel', 'Keyboard', 1, 1200],

```

```
['ORD-004', 'David Singh', 'Monitor', 1, 18000],  
]
```

```
with open('sales.csv', 'w', newline='') as file:
```

```
    writer = csv.writer(file)
```

```
    writer.writerows(sales_records)
```

```
print('Sales report saved to sales.csv')
```

Figure 4.2 — Writing sales records to a CSV file using csv.writer



Pro Tip

The **csv module** is always the preferred approach over manually splitting lines. Even if your current CSV file is simple, using the module ensures your code will handle edge cases correctly as the data grows and changes.

ETIHANS
CLOUD & AI ACADEMY



Chapter 5

Working with DictReader and DictWriter

So far, you have worked with `csv.reader` and `csv.writer` where each row is handled as a **list**. This means you access data using index numbers like `row[0]` or `row[2]`. While this works, it can be confusing — especially when working with files that have many columns.

Python's `csv` module offers a better solution: **DictReader** and **DictWriter**. These tools allow you to work with rows as **dictionaries**, where each value is accessed by its column name instead of a number.

5.1 csv.DictReader

csv.DictReader

A class in Python's `csv` module that reads each row of a CSV file as an ordered dictionary. The keys of the dictionary are taken from the header row of the CSV file, and the values are the data in each row.

Let us see `DictReader` in action using the same **students.csv** file:

```
import csv

# Reading with DictReader — rows come back as dictionaries
with open('students.csv', 'r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row)
```

Figure 5.1 — Using `csv.DictReader` to read rows as dictionaries

Output

```
{'Name': 'Alice Sharma', 'Age': '21', 'Course': 'Computer Science', 'Grade': 'A'}
{'Name': 'Bob Mehta', 'Age': '22', 'Course': 'Data Science', 'Grade': 'B'}
{'Name': 'Carol Patel', 'Age': '20', 'Course': 'Computer Science', 'Grade': 'A'}
{'Name': 'David Singh', 'Age': '23', 'Course': 'AI & ML', 'Grade': 'A'}
{'Name': 'Eva Nair', 'Age': '21', 'Course': 'Data Science', 'Grade': 'B'}
```

Now each row is a dictionary. To access any value, you use the **column name** as the key — which is much more readable than using an index number.


```
import csv

# Accessing data by column name using DictReader
with open('students.csv', 'r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        name = row['Name']    # Access by column name — clear and readable
        age = int(row['Age']) # Convert to int for calculation
        course = row['Course']
        grade = row['Grade']

    print(f'{name} (Age {age}) is studying {course} and received Grade {grade}')
```

Figure 5.2 — Accessing column values by name with DictReader

Output

Alice Sharma (Age 21) is studying Computer Science and received Grade A
Bob Mehta (Age 22) is studying Data Science and received Grade B
Carol Patel (Age 20) is studying Computer Science and received Grade A
David Singh (Age 23) is studying AI & ML and received Grade A
Eva Nair (Age 21) is studying Data Science and received Grade B

5.2 csv.DictWriter

csv.DictWriter

A class in Python's csv module that writes rows to a CSV file from dictionaries. You specify the column names (fieldnames) when creating the writer, and then pass a dictionary for each row you want to write.

```
import csv

# Student records stored as dictionaries
students = [
    {'Name': 'Alice Sharma', 'Age': 21, 'Course': 'Computer Science', 'Grade': 'A'},
    {'Name': 'Bob Mehta', 'Age': 22, 'Course': 'Data Science', 'Grade': 'B'},
    {'Name': 'Carol Patel', 'Age': 20, 'Course': 'Computer Science', 'Grade': 'A'},
]
```

```
# Define the column order
fieldnames = ['Name', 'Age', 'Course', 'Grade']

with open('output_students.csv', 'w', newline='') as file:
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()    # Automatically writes the header row
    writer.writerows(students) # Writes all the dictionary rows

print('Student file written successfully!')
```

Figure 5.3 — Writing data using csv.DictWriter**Note**

writer.writeheader() automatically writes the column names as the first row of the CSV. You do not need to write the header row manually — DictWriter does it for you.

Make sure the keys in every dictionary match the **fieldnames** list exactly. If a dictionary has a key that is not in fieldnames, Python will raise an error.

5.3 When to Use Dict vs Regular Reader/Writer

Situation	Use csv.reader / csv.writer	Use DictReader / DictWriter
Number of columns	File has very few columns (2–3)	File has many columns
Column access method	You prefer index-based access (row[0])	You prefer name-based access (row['Name'])
Readability	Simple scripts where clarity isn't critical	Professional code, team projects, or teaching
Data source	Data already in lists format	Data stored in dictionaries (very common in APIs and databases)
Header row	You manually skip it with next()	DictReader/DictWriter handles headers automatically

**Pro Tip**

In professional Python development, **DictReader** and **DictWriter** are preferred because the code is more readable and maintainable. When someone reads your code six months later, **row['Salary']** is immediately clear — **row[3]** is not.

ETHANS
CLOUD & AI ACADEMY



Chapter 6

Handling Delimiters and Headers

In the real world, CSV files do not always use commas as their separator. You may receive data files that use semicolons, tabs, pipes, or other characters to separate values. Additionally, some files may not have a header row, or the header may need to be customized.

In this chapter, you will learn how to handle these variations confidently.

6.1 What is a Delimiter?

Delimiter	The character used to separate individual values (fields) within a row of a CSV file. The default delimiter in a standard CSV file is a comma (,), but other characters are also commonly used.
-----------	---

Here is a comparison of different delimiters you may encounter:

Delimiter	Symbol	Example Row	Common Use Case
Comma	,	Alice,21,CSE,A	Standard CSV files (most common)
Semicolon	;	Alice;21;CSE;A	European CSV files (where , is the decimal point)
Tab	\t	Alice\t21\tCSE\tA	TSV (Tab-Separated Values) files — exported from Excel
Pipe		Alice 21 CSE A	Legacy database exports and some banking systems
Space		Alice 21 CSE A	Some scientific and log data files

6.2 Custom Delimiters in csv.reader

Python's csv module makes it easy to handle different delimiters. You simply pass the **delimiter** parameter when creating the reader or writer object.

Let us say you receive a file called **inventory.csv** that uses semicolons instead of commas:

```
Product;Category;Price;Stock
Laptop;Electronics;55000;120
Notebook;Stationery;40;5000
```

```
Headphones;Electronics;1200;300
```

Figure 6.1 — A semicolon-delimited CSV file (inventory.csv)

```
import csv

# Reading a file with semicolon delimiter
with open('inventory.csv', 'r') as file:
    reader = csv.reader(file, delimiter=';') # Specify the delimiter

    for row in reader:
        print(row)
```

Figure 6.2 — Reading a semicolon-delimited file with custom delimiter

Output

```
['Product', 'Category', 'Price', 'Stock']
['Laptop', 'Electronics', '55000', '120']
['Notebook', 'Stationery', '40', '5000']
['Headphones', 'Electronics', '1200', '300']
```

The same approach works with tab-delimited files (TSV), which are very commonly exported from Microsoft Excel:

```
import csv

# Reading a tab-separated file (.tsv or .txt)
with open('data.tsv', 'r') as file:
    reader = csv.reader(file, delimiter='\t') # \t is the tab character

    for row in reader:
        print(row)
```

Figure 6.3 — Reading a tab-delimited file

6.3 Handling Headers

The first row of most CSV files is a **header row** — it contains column names, not data. How you handle this header depends on your task.

There are three common scenarios:

Scenario 1: Read the header and print it separately

```
import csv

with open('students.csv', 'r') as file:
    reader = csv.reader(file)

    header = next(reader)    # Read the header row
    print('Columns available:', header)

    for row in reader:      # Now reads only data rows
        print(row)
```

Figure 6.4 — Reading the header row separately

Scenario 2: Store the header for later use

```
import csv

with open('students.csv', 'r') as file:
    reader = csv.reader(file)
    header = next(reader)    # Save the header

    data_rows = list(reader) # Store all data rows as a list

print('Header:', header)
print('Total records:', len(data_rows))
print('First record:', data_rows[0])
```

Figure 6.5 — Saving header and data rows separately

Output

Header: ['Name', 'Age', 'Course', 'Grade']

Total records: 5

First record: ['Alice Sharma', '21', 'Computer Science', 'A']

6.4 Skipping the Header Row

Sometimes you just want to process the data and ignore the header completely. The **next()** function advances the reader past the first row without storing it:

```
import csv

total_age = 0
count = 0

with open('students.csv', 'r') as file:
    reader = csv.reader(file)
    next(reader) # Skip the header row — we do not need it

    for row in reader:
        total_age += int(row[1]) # Sum up the ages
        count += 1

average_age = total_age / count
print(f'Number of students: {count}')
print(f'Average age: {average_age:.1f} years')
```

Figure 6.6 — Skipping the header row and calculating average age

Output

Number of students: 5
Average age: 21.4 years



Writing Files with Custom Delimiters

You can also write CSV files with a custom delimiter using the same parameter in `csv.writer`:

```
with open('output.csv', 'w', newline='') as file:
    writer = csv.writer(file, delimiter=';') # Write with semicolon separator
    writer.writerow(['Name', 'Age', 'Course'])
    writer.writerow(['Alice', 21, 'Computer Science'])
```

This is useful when you need to create files that will be imported into European tools or legacy systems.

Chapter 7

Dealing with CSV Datasets

In the previous chapters, you worked with small CSV files that you created yourself. In the real world, you will often work with **datasets** — large CSV files containing hundreds or thousands of records downloaded from databases, government portals, or business systems.

In this chapter, you will learn how to load, explore, filter, sort, and process real CSV datasets using Python.

7.1 What is a Dataset?

Dataset

A structured collection of data, typically stored in a file or database, used for analysis, reporting, or processing. In the context of CSV files, a dataset is a CSV file with many rows of real-world data — such as sales records, employee data, or weather measurements.

For this chapter, we will work with the following sample dataset: **sales_data.csv**, which represents monthly sales records of a retail company.

```
OrderID,CustomerName,Product,Category,Quantity,UnitPrice,City,Month
1001,Alice Sharma,Laptop,Electronics,1,55000,Ahmedabad,January
1002,Bob Mehta,Notebook,Stationery,10,40,Mumbai,January
1003,Carol Patel,Headphones,Electronics,2,1200,Pune,February
1004,David Singh,Laptop,Electronics,1,55000,Bangalore,February
1005,Eva Nair,Pen Drive,Electronics,3,450,Chennai,March
1006,Farhan Khan,Keyboard,Electronics,1,1200,Hyderabad,March
1007,Geeta Rao,Notebook,Stationery,20,40,Ahmedabad,March
1008,Harish Patil,Monitor,Electronics,1,18000,Mumbai,April
1009,Isha Joshi,Headphones,Electronics,1,1200,Pune,April
1010,Jay Shah,Laptop,Electronics,2,55000,Ahmedabad,April
```

Figure 7.1 — Sample sales_data.csv dataset

7.2 Loading a Real Dataset

The first step when working with any dataset is to load it and explore its structure — how many rows it has, what columns are available, and what the data looks like.


```
import csv

# Load the dataset and display basic information
with open('sales_data.csv', 'r') as file:
    reader = csv.DictReader(file)

    rows = list(reader)    # Load all rows into a list

# Basic information about the dataset
print(f'Total orders: {len(rows)}')
print(f'Columns: {list(rows[0].keys())}')
print()
print('--- First 3 Records ---')
for row in rows[:3]:      # Display first 3 records
    print(row)
```

Figure 7.3 — Loading and exploring a CSV dataset

Output

Total orders: 10

Columns: ['OrderID', 'CustomerName', 'Product', 'Category', 'Quantity', 'UnitPrice', 'City', 'Month']

--- First 3 Records ---

{'OrderID': '1001', 'CustomerName': 'Alice Sharma', 'Product': 'Laptop', ...}

{'OrderID': '1002', 'CustomerName': 'Bob Mehta', 'Product': 'Notebook', ...}

{'OrderID': '1003', 'CustomerName': 'Carol Patel', 'Product': 'Headphones', ...}

7.3 Filtering Rows Based on Conditions

One of the most useful operations on a dataset is **filtering** — selecting only the rows that match a specific condition. For example: show all orders from Ahmedabad, or show all orders for Electronics products.

```
import csv

# Filter: Show only Electronics category orders
with open('sales_data.csv', 'r') as file:
    reader = csv.DictReader(file)
```

```

electronics_orders = []

for row in reader:
    if row['Category'] == 'Electronics':    # Apply filter condition
        electronics_orders.append(row)

print(f'Total Electronics orders: {len(electronics_orders)}')
print()
for order in electronics_orders:
    total = int(order['Quantity']) * int(order['UnitPrice'])
    print(f"{order['CustomerName']:20} | {order['Product']:12} | Rs. {total:>8,}")

```

Figure 7.4 — Filtering rows by category

Output

Total Electronics orders: 8

Alice Sharma	Laptop	Rs. 55,000
Carol Patel	Headphones	Rs. 2,400
David Singh	Laptop	Rs. 55,000
Eva Nair	Pen Drive	Rs. 1,350
Farhan Khan	Keyboard	Rs. 1,200
Harish Patil	Monitor	Rs. 18,000
Isha Joshi	Headphones	Rs. 1,200
Jay Shah	Laptop	Rs. 110,000

You can also apply **multiple conditions** using **and** / **or**:

```

import csv

# Filter: Electronics orders from Ahmedabad only
with open('sales_data.csv', 'r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        if row['Category'] == 'Electronics' and row['City'] == 'Ahmedabad':
            total = int(row['Quantity']) * int(row['UnitPrice'])
            print(f"{row['CustomerName']} ordered {row['Product']} — Rs. {total:,}")

```

Figure 7.5 — Filtering with multiple conditions

Output

Alice Sharma ordered Laptop — Rs. 55,000
 Geeta Rao ordered Notebook — Rs. 800
 Jay Shah ordered Laptop — Rs. 1,10,000

7.4 Sorting and Aggregating Data

After loading a dataset into a list, you can use Python's built-in functions to sort and aggregate the data.

Sorting a Dataset:

```
import csv

with open('sales_data.csv', 'r') as file:
    reader = csv.DictReader(file)
    rows = list(reader)

# Sort by UnitPrice in descending order (highest price first)
sorted_rows = sorted(rows, key=lambda row: int(row['UnitPrice']), reverse=True)

print('Products sorted by Unit Price (highest first):' )
print(f"{'Product':<15} {'Unit Price':>12}")
print('-' * 30)
for row in sorted_rows:
    print(f"{'row['Product']':<15} Rs. {int(row['UnitPrice']):>8,}")
```

Figure 7.6 — Sorting the dataset by unit price

Output

Products sorted by Unit Price (highest first):

Product	Unit Price
Laptop	Rs. 55,000
Laptop	Rs. 55,000
Laptop	Rs. 55,000
Monitor	Rs. 18,000
Headphones	Rs. 1,200
...	

Aggregating Data — Calculating Total Sales:

```
import csv

with open('sales_data.csv', 'r') as file:
    reader = csv.DictReader(file)
    rows = list(reader)

# Calculate total revenue per product
product_totals = {}

for row in rows:
    product = row['Product']
    revenue = int(row['Quantity']) * int(row['UnitPrice'])

    if product not in product_totals:
        product_totals[product] = 0
    product_totals[product] += revenue

print('Total Revenue by Product:')
print(f'{"Product":<15} {"Total Revenue":>15}')
print('-' * 32)
for product, total in sorted(product_totals.items(), key=lambda x: x[1], reverse=True):
    print(f'{product:<15} Rs. {total:>10},')
```

*Figure 7.7 — Aggregating total revenue by product***Output**

Total Revenue by Product:

Product	Total Revenue
Laptop	Rs. 1,65,000
Monitor	Rs. 18,000
Headphones	Rs. 3,600
Keyboard	Rs. 1,200
Pen Drive	Rs. 1,350
Notebook	Rs. 1,200

7.5 Writing a Cleaned Dataset to a New File

A very common workflow in data processing is:

- Load the original CSV dataset
- Filter or transform the data
- Write the processed results to a new CSV file

This is sometimes called a **data pipeline** or an **ETL process** (Extract, Transform, Load). Here is a complete example:

```
import csv

# Step 1: Load the original dataset
with open('sales_data.csv', 'r') as infile:
    reader = csv.DictReader(infile)
    all_rows = list(reader)

# Step 2: Filter — keep only Electronics orders above Rs. 5,000 total value
filtered = []
for row in all_rows:
    total_value = int(row['Quantity']) * int(row['UnitPrice'])
    if row['Category'] == 'Electronics' and total_value >= 5000:
        row['TotalValue'] = total_value # Add a new calculated column
        filtered.append(row)

# Step 3: Write the filtered + enriched data to a new file
fieldnames = ['OrderID', 'CustomerName', 'Product', 'Quantity', 'UnitPrice', 'TotalValue', 'City', 'Month']

with open('high_value_electronics.csv', 'w', newline='') as outfile:
    writer = csv.DictWriter(outfile, fieldnames=fieldnames, extrasaction='ignore')
    writer.writeheader()
    writer.writerows(filtered)

print(f'Processed {len(filtered)} high-value electronics orders.')
print('Saved to: high_value_electronics.csv')
```

Figure 7.8 — Complete data pipeline: Load → Filter → Write

Output

Processed 4 high-value electronics orders.
Saved to: high_value_electronics.csv

Real-World Application — Data Cleaning Pipeline

This type of workflow is used daily in data engineering and analytics jobs. For example:

- A retail company downloads daily sales data from their POS system as a CSV file.
- A Python script filters only completed orders, calculates totals and discounts, and removes returns.
- The cleaned CSV is saved and automatically imported into a reporting dashboard.

This process — often running hundreds of times a day — is exactly what you have just built in Figure 7.8.

ETIHANS
CLOUD & AI ACADEMY



Chapter Summary

In this chapter, you have covered all the essential skills for working with CSV files using Python. Here is a recap of the key concepts:

Topic	Key Takeaway
Introduction to CSV Files	CSV files are plain-text files that store tabular data using commas (or other delimiters) to separate values. They are used universally for data exchange.
Reading CSV Files	Use <code>csv.reader</code> with the <code>open()</code> function to read a CSV file line by line. Each row is returned as a Python list. Use <code>next(reader)</code> to skip the header.
Writing CSV Files	Use <code>csv.writer</code> to write rows to a CSV file. Open in 'w' mode to create/overwrite, and 'a' mode to append. Always include <code>newline=""</code> on Windows.
Using the csv Module	The <code>csv</code> module handles edge cases (commas inside text, line endings, special characters) automatically — always prefer it over manual string splitting.
DictReader and DictWriter	<code>DictReader</code> reads each row as a dictionary (key = column name). <code>DictWriter</code> writes from dictionaries. Both are more readable and professional than index-based access.
Delimiters and Headers	Use the <code>delimiter</code> parameter to handle files with semicolons, tabs, or pipes. Use <code>next(reader)</code> to skip or store the header row.
Working with Datasets	Load CSV datasets into a list, then use Python logic to filter, sort, and aggregate. Use <code>DictWriter</code> to save processed results back to a new CSV file.

© Python Programming Course — Confidential Course Material